

Implementation of Programming Languages Lab: CS348
3rd Year CSE

Assignment - 4: Parser for nanoC
Assign Date: April 7, 2026

Submit Date: 23:59, April 24, 2026

1 Preamble – nanoC

This assignment follows the phase structure grammar specification of C language from the International Standard ISO/IEC 9899:1999 (E). To keep the assignment within our required scope, we have chosen a subset of the specification as given below. We shall refer to this language as nanoC.

The lexical specification of nanoC, also taken and abridged from the Standard, has already been discussed in Assignment 3.

The phase structure grammar specification is written using the common notation of language specifications as discussed in the last assignment.

2 Phrase Structure Grammar of C

1. Expressions

primary-expression:
 identifier
 constant
 string-literal
 (*expression*)
postfix-expression:
 primary-expression
 postfix-expression [*expression*]
 postfix-expression (*argument-expression-list*_{opt})
 postfix-expression ++
 postfix-expression --
argument-expression-list:
 assignment-expression
 argument-expression-list , *assignment-expression*
unary-expression:
 postfix-expression
 ++ *unary-expression*
 -- *unary-expression*
unary-operator: one of

 & * + - ~ !

multiplicative-expression:
 unary-expression
 multiplicative-expression * *cast-expression*
 multiplicative-expression / *cast-expression*
 multiplicative-expression % *cast-expression*

additive-expression:
multiplicative-expression
additive-expression + *multiplicative-expression*
additive-expression - *multiplicative-expression*
shift-expression:
additive-expression
shift-expression << *additive-expression*
shift-expression >> *additive-expression*
relational-expression:
shift-expression
relational-expression < *shift-expression*
relational-expression > *shift-expression*
relational-expression <= *shift-expression*
relational-expression >= *shift-expression*
equality-expression:
relational-expression
equality-expression == *relational-expression*
equality-expression != *relational-expression*
AND-expression:
equality-expression
AND-expression & *equality-expression*
exclusive-OR-expression:
AND-expression
exclusive-OR-expression ^ *AND-expression*
inclusive-OR-expression:
exclusive-OR-expression
inclusive-OR-expression | *exclusive-OR-expression*
logical-AND-expression:
inclusive-OR-expression
logical-AND-expression && *inclusive-OR-expression*
logical-OR-expression:
logical-AND-expression
logical-OR-expression || *logical-AND-expression*
conditional-expression:
logical-OR-expression
logical-OR-expression ? *expression* : *conditional-expression*
assignment-expression:
conditional-expression
unary-expression *assignment-operator* *assignment-expression*
assignment-operator: one of

 = * = / = % = + = - = < = > = & = ^ = | =

expression:
assignment-expression
expression , *assignment-expression*
constant-expression:
conditional-expression

2. Declarations

declaration:
declaration-specifiers *init-declarator-list*_{opt} ;
declaration-specifiers:
storage-class-specifier *declaration-specifiers*_{opt}
type-specifier *declaration-specifiers*_{opt}
init-declarator-list:
init-declarator
init-declarator-list , *init-declarator*
init-declarator:
declarator
declarator = *initializer*
storage-class-specifier:
static

type-specifier:
 void
 char
 short
 int
 long
 float
 double
 signed
 unsigned
 _Bool
declarator:
 direct-declarator
direct-declarator:
 identifier
 (*declarator*)
 direct-declarator [*assignment-expression*_{opt}]
 direct-declarator
 direct-declarator (*parameter-type-list*)
 direct-declarator (*identifier-list*_{opt})
parameter-type-list:
 parameter-list
 parameter-list , ...
parameter-list:
 parameter-declaration
 parameter-list , *parameter-declaration*

parameter-declaration:
 declaration-specifiers *declarator*
 declaration-specifiers
identifier-list:
 identifier
 identifier-list , *identifier*
initializer:
 assignment-expression
 { *initializer-list* }
 { *initializer-list* , }
initializer-list:
 *designation*_{opt} *initializer*
 initializer-list , *designation*_{opt} *initializer*
designation:
 designator-list =
designator-list:
 designator
 designator-list *designator*
designator:
 [*constant-expression*]

3. Statements

statement:
 labeled-statement
 compound-statement
 expression-statement
 selection-statement
 iteration-statement
 jump-statement
labeled-statement:
 identifier : *statement*
 case *constant-expression* : *statement*
 default : *statement*
compound-statement:
 { *block-item-list*_{opt} }

block-item-list:
 block-item
 block-item-list block-item
block-item:
 declaration
 statement
expression-statement:
 *expression*_{opt} ;
selection-statement:
 if (*expression*) *statement*
 if (*expression*) *statement* **else** *statement*
iteration-statement:
 while (*expression*) *statement*
 do *statement* **while** (*expression*) ;
 for (*expression*_{opt} ; *expression*_{opt} ; *expression*_{opt}) *statement*
 for (*declaration* *expression*_{opt} ; *expression*_{opt}) *statement*
jump-statement:
 continue ;
 break ;
 return *expression*_{opt} ;

3 The Assignment

- (a) Write a bison specification for defining the tokens of **nanoC** and generate the required `y.tab.h` file.
- (b) Write a bison specification for the language of **nanoC** using the above phase structure grammar. Use the flex specification that you had developed for Assignment 3 (if required, you may fix your flex specification).
- (c) While writing the bison specification, you may need to make some changes to the grammar. For example, some non-terminals like

argument-expression-list_{opt}

are shown as optional on the right-hand-side as:

postfix-expression:

postfix-expression (*argument-expression-list_{opt}*)

One way to handle them would be to introduce a new non-terminal, *argument-expression-list-opt*, and a pair of new productions:

argument-expression-list-opt:

argument-expression-list

ϵ

and change the above rule as:

postfix-expression:

postfix-expression (*argument-expression-list-opt*)

- (d) Names of your `.l` and `.y` files should be `a4_roll.l` and `a4_roll.y` respectively.
- (e) Prepare a Makefile to compile the specifications and generate the lexer and the parser.
- (f) Prepare a test input file `a4_roll_test.nc` that will test all the rules that you have coded.
- (g) Prepare a zip file with the name `a4_roll.zip` containing all the files and upload to Moodle.

4 Marks Distribution

- (a) Specifications and testing: **60%**
- (b) File Names and makefile: **20%**
- (c) ReadMe & Test file: **20%**

5 Commands:

- `flex a4_roll.l`
- `yacc -d a4_roll.y`
- `gcc lex.yy.c y.tab.c`
- `./a.out < a4_roll_test.nc`